ConnectSphere Auth Service - Spring Boot Concepts Guide

This document is a complete study guide for the Spring Boot concepts used in the `auth-service`. The goal is not only to say what each concept means, but also where it is used in this project, why it is needed, and how it works internally inside Spring Boot and the wider Spring ecosystem.

Scope of this guide
Service: `auth-service`
Base package: `com.connectsphere.auth`
Focus: all Spring Boot and Spring ecosystem concepts actually used in this service
Coverage style: definition + project usage + internal working

1. What This Service Uses from Spring

Even though we casually say “Spring Boot concepts,” this auth service actually uses several connected layers:

Technology Area	What it contributes in this auth service
Spring Boot	Application startup, dependency starters, auto-configuration, profile-based configuration, Actuator, test bootstrapping.
Spring Framework Core	Dependency injection, configuration classes, bean lifecycle, component scanning.
Spring MVC	REST controllers, request mapping, request/response conversion, validation integration, exception handling.
Spring Security	Authentication manager, security filter chain, password encoding, custom user loading, protected endpoints.
Spring Data JPA	Entity mapping, repositories, database persistence, JPQL queries, transactions.

Technology Area	What it contributes in this auth service
Spring Boot Test	Integration testing with a full application context and MockMvc.

2. Starter Dependencies and Why They Matter

One of the first big Spring Boot concepts in this service is the use of **starter dependencies**. A starter is a curated dependency bundle that brings the common libraries, defaults, and auto-configuration hooks for a given area.

Dependency	Why this auth service needs it	What Spring Boot auto-configures because of it
`spring-boot-starter-web`	Needed for REST endpoints such as register, login, refresh, profile, and search.	Embedded web server, DispatcherServlet, JSON conversion, request mapping infrastructure.
`spring-boot-starter-validation`	Needed so request DTOs can use validation annotations like `@NotBlank`, `@Email`, and `@Size`.	Bean Validation integration with controller argument resolution.
`spring-boot-starter-data-jpa`	Needed so users and revoked tokens can be stored in a relational database through entities and repositories.	EntityManagerFactory, transaction manager, repository scanning, JPA infrastructure.
`spring-boot-starter-security`	Needed for login verification, route protection, password hashing, security context management, and filter integration.	Security filter infrastructure, authentication support, default security beans.
`spring-boot-starter-oauth2-client`	Included because the case study requires future Google/GitHub login support.	OAuth client registration and related support when the `oauth` profile is enabled.
`spring-boot-starter-actuator`	Needed to expose operational endpoints such as health checks.	Actuator endpoints and management infrastructure.

Dependency	Why this auth service needs it	What Spring Boot auto-configures because of it
`spring-boot-starter-test`	Needed to test the auth service with a real Spring Boot application context.	Spring test support, context loading, assertions, test bootstrapping.

Internal Working of Starters

A starter itself is not the feature. It is a dependency package that pulls in the right libraries, and those libraries expose classes that Spring Boot recognizes through **conditional auto-configuration**. During startup, Spring Boot checks the classpath, the configuration files, and the existing beans. Then it decides what beans to create automatically.

```
Application starts
  -> Spring Boot inspects classpath
  -> finds spring-web classes, spring-security classes, JPA classes, validation classes
  -> loads matching auto-configuration classes
  -> creates beans if conditions match and no custom bean overrides them
  -> application becomes ready with far less manual setup
```

3. @SpringBootApplication

The file `AuthServiceApplication.java` uses `@SpringBootApplication`. This is one of the most central Spring Boot concepts.

Definition

`@SpringBootApplication` is a convenience annotation that combines:

- `@Configuration` - marks the class as a source of bean definitions
- `@EnableAutoConfiguration` - lets Spring Boot auto-configure beans based on the classpath and config
- `@ComponentScan` - scans the package and subpackages for Spring-managed components

Why It Is Needed Here

Without it, this service would need a much larger amount of manual setup. We would have to explicitly configure component scanning, import auto-configuration classes, and bootstrap the

application context ourselves.

How It Works Internally

```
SpringApplication.run(AuthServiceApplication.class, args)
  -> creates ApplicationContext
  -> uses AuthServiceApplication as the primary source
  -> component scan starts from package com.connectsphere.auth
  -> discovers controllers, services, repositories, config classes, filters
  -> auto-configuration loads infrastructure beans
  -> web server starts and beans become injectable
```

4. Component Scanning and Spring Beans

A **bean** is an object managed by Spring. Instead of creating these objects manually with `new` all over the code, Spring creates them, stores them in the application context, wires them together, and manages their lifecycle.

@Service

Used on `AuthServiceImpl`, `JwtTokenService`, and `CustomUserDetailsService`.

Marks business or infrastructure classes as Spring-managed service beans.

@RestController

Used on `AuthResource`.

Marks the class as a web controller whose methods return data directly as HTTP responses, usually JSON.

@Configuration

Used on `SecurityConfig`.

Marks the class as a source of bean definitions.

@Component

Used on `JwtAuthenticationFilter`.

Marks the filter as a generic Spring-managed bean so it can be injected into the security configuration.

Internal Working

During component scanning, Spring searches the `com.connectsphere.auth` package tree because that is where the main application class is located. When it finds classes with stereotype

annotations such as `@Service` or `@RestController`, it registers them as beans in the application context.

5. Dependency Injection and Constructor Injection

Definition

Dependency Injection means a class does not build its own collaborators. Instead, Spring provides them.

Where It Is Used Here

- `AuthResource` receives `AuthService` through its constructor
- `AuthServiceImpl` receives `UserRepository`, `PasswordEncoder`, `AuthenticationManager`, and `JwtTokenService`
- `SecurityConfig` receives `JwtAuthenticationFilter` and `UserDetailsService`
- `JwtAuthenticationFilter` receives `JwtTokenService` and `CustomUserDetailsService`

Why Constructor Injection Is Better

- Dependencies are explicit
- The object cannot exist in a half-initialized state
- It is easier to test and reason about
- It works cleanly with final fields

Internal Working

```
Spring creates bean A
  -> sees constructor needs bean B and bean C
  -> looks them up in application context
  -> creates them first if necessary
  -> passes them into bean A constructor
  -> stores fully initialized bean A
```

6. Configuration Files, Profiles, and Configuration Properties

6.1 application.yml

Spring Boot loads `application.yml` automatically and binds its values into the environment. This auth service uses it for server port, datasource config, JPA settings, Actuator exposure, and JWT settings.

6.2 Profiles

Profiles let one Spring Boot app behave differently in different environments.

- `application.yml` = default local/test behavior
- `application-mysql.yml` = switch persistence to MySQL
- `application-oauth.yml` = enable social-login client configuration

Why Profiles Are Needed

The auth service should not need different code for H2, MySQL, and OAuth-enabled modes. Profiles let us switch behavior with configuration only.

6.3 @ConfigurationProperties with a Record

`JwtProperties` uses `@ConfigurationProperties(prefix = "app.jwt")`. This is a very important Spring Boot concept.

Definition

`@ConfigurationProperties` binds a configuration section into a strongly typed Java object.

Why It Is Better Than Reading Strings Manually

- Type safety: durations become `Duration`, not raw strings everywhere
- Validation support through annotations like `@NotBlank` and `@NotNull`
- Cleaner code in services

Why a Record Is Used Here

A Java `record` is ideal for immutable configuration data. Spring Boot can construct the record directly from YAML values, and the record naturally represents “fixed configuration, not changing state.”

Internal Working

```
Spring Boot reads application.yml
  -> finds app.jwt section
  -> sees JwtProperties is registered with @EnableConfigurationProperties
  -> binds issuer, secret, accessTokenTtl, refreshTokenTtl
  -> validates fields
  -> stores JwtProperties as a bean
  -> JwtTokenService receives it through constructor injection
```

6.4 @EnableConfigurationProperties

The main application class uses `@EnableConfigurationProperties(JwtProperties.class)`.

Definition

This tells Spring Boot to register the given configuration-properties type as a managed bean and bind external configuration into it.

Why It Is Needed Here

Without it, `JwtProperties` would not automatically become available for injection into `JwtTokenService`.

6.5 @Bean Methods

`SecurityConfig` defines bean methods such as `securityFilterChain()`, `authenticationProvider()`, `passwordEncoder()`, and `authenticationManager()`.

Definition

A method annotated with `@Bean` tells Spring, "the object returned by this method should be stored in the application context and reused as a managed bean."

Why It Is Needed Here

Some objects in the auth service are not discovered just by component scanning. The security chain and custom authentication infrastructure must be created intentionally inside configuration code, and `@Bean` is how that happens.

7. Spring MVC Concepts Used in the Auth Service

7.1 @RestController

`@RestController` means this class handles HTTP requests and returns serialized response bodies directly. It is effectively `@Controller` plus `@ResponseBody`.

7.2 @RequestMapping and Method-Level Mappings

`AuthResource` uses class-level `@RequestMapping({"/api/v1/auth", "/auth"})` and method-level mappings like `@PostMapping("/login")`, `@GetMapping("/profile")`, and `@PatchMapping("/password")`.

This is needed so one controller can expose multiple routes cleanly while keeping the endpoint URLs readable and organized.

Internal Working of Request Mapping

```
HTTP request arrives
  -> DispatcherServlet receives it
  -> HandlerMapping finds matching controller method
  -> argument resolvers prepare method parameters
  -> controller method runs
  -> return value is converted into HTTP response
```

7.3 @RequestBody

Used for request DTOs like register, login, refresh, logout, password change, and token validation. Spring reads the incoming JSON and converts it into the matching Java record.

7.4 ResponseEntity

`ResponseEntity` is used when the controller needs explicit control over the HTTP response status and body. For example, register returns HTTP 201 Created, while login returns HTTP 200 OK.

7.5 @RequestParam and @PathVariable

- `@RequestParam("query")` is used for search
- `@PathVariable` is used for the by-email endpoint

These concepts let Spring extract URL query values and URL path values and inject them as method arguments.

7.6 Principal

Principal represents the authenticated identity of the current request. In this service, after the JWT filter succeeds, the controller can ask for `Principal principal`, and `principal.getName()` returns the authenticated email.

Why Principal Is Needed Here

Without Principal, the profile, password, and deactivate endpoints would not know which logged-in user is making the request.

Internal Working

```
JWT filter authenticates request
  -> builds UsernamePasswordAuthenticationToken
  -> stores it in SecurityContextHolder
  -> Spring MVC later resolves Principal from the security context
  -> controller receives Principal as a method parameter
```

8. Validation Concepts

8.1 @Valid

`@Valid` on controller arguments tells Spring to validate the incoming DTO before the method logic runs.

8.2 Jakarta Validation Annotations

The auth service uses annotations like:

- `@NotBlank`
- `@NotNull`
- `@Email`
- `@Size`

Why Validation Is Needed

- Reject bad input early
- Keep service logic cleaner
- Prevent malformed data from reaching persistence and security logic

Internal Working

```
Request JSON arrives
  -> Spring converts it into DTO
  -> Bean Validation checks annotations
  -> if invalid, MethodArgumentNotValidException is thrown
  -> GlobalExceptionHandler formats the error response
```

9. Exception Handling with @RestControllerAdvice

`GlobalExceptionHandler` uses `@RestControllerAdvice`. This is a Spring MVC mechanism for centralized exception handling.

Definition

`@RestControllerAdvice` applies exception-handling methods globally across controllers and returns serialized response bodies.

Why It Is Needed

- Controllers stay clean
- Error responses stay consistent
- Business logic can throw meaningful exceptions without worrying about HTTP formatting

Internal Working

```
Controller or service throws exception
  -> exception bubbles up through Spring MVC
  -> ExceptionHandler methods are checked
  -> matching handler builds ResponseEntity
  -> HTTP response is sent with status and body
```

10. Spring Data JPA Concepts

10.1 @Entity and Table Mapping

`User` and `RevokedToken` are annotated with `@Entity`. This tells JPA that these classes should be mapped to database tables.

10.2 @Id, @Column, @Enumerated

- `@Id` marks the primary key field
- `@Column` customizes database column mapping
- `@Enumerated(EnumType.STRING)` stores enum values as readable strings

Why Entities Are Needed

The auth service needs permanent storage for users and revoked token ids. Entities are the bridge between Java objects and relational tables.

10.3 JPA Lifecycle Hooks

The `User` entity uses `@PrePersist` and `@PreUpdate`.

- `@PrePersist` sets id and timestamps before first save
- `@PreUpdate` refreshes the updated timestamp on modification

This keeps repetitive audit logic out of the service layer.

10.4 Repository Interfaces

`UserRepository` and `RevokedTokenRepository` extend `JpaRepository`.

Definition

`JpaRepository` is a Spring Data interface that provides ready-made CRUD operations and integrates with JPA automatically.

Why It Is Needed

Without it, this service would need a much heavier DAO layer with manual entity manager code.

10.5 Derived Query Methods and JPQL

Spring Data JPA can create queries from method names such as:

- `findByEmail`
- `findByUsername`

- `existsByEmail`
- `deleteByExpiresAtBefore`

It also supports custom JPQL queries through `@Query`, which is how `searchByUsername` is implemented.

10.6 @Transactional

`AuthServiceImpl` is annotated with `@Transactional`, and some methods use `@Transactional(readOnly = true)`.

Definition

A transaction wraps a unit of database work so that reads and writes stay consistent.

Why It Is Needed Here

- Registration writes a full user record atomically
- Password changes and deactivation update data safely
- Read-only methods communicate intent and may optimize persistence behavior

Internal Working

```
Controller calls service method
  -> Spring proxy intercepts call
  -> transaction starts before method body
  -> repository operations run inside transaction
  -> transaction commits on success or rolls back on failure
```

11. Spring Security Concepts

This auth service uses Spring Security heavily. It does not just “protect endpoints”; it also verifies credentials, stores the authenticated user in the request context, and provides the extension points used by JWT authentication.

11.0 @EnableMethodSecurity

`SecurityConfig` uses `@EnableMethodSecurity`.

Definition

This annotation turns on method-level security support in Spring, enabling annotations such as `@PreAuthorize` if the service needs them.

Why It Is Included Here

Even though the current auth service mainly secures endpoints through the filter chain, enabling method security keeps the service ready for future fine-grained authorization checks in service methods.

11.1 SecurityFilterChain

`SecurityFilterChain` defines how requests are secured.

Why It Is Needed

- To declare which routes are public
- To declare which routes require authentication
- To insert the JWT filter in the right place
- To disable session-based behavior for a stateless API

Internal Working

When an HTTP request enters the application, it passes through the security filter chain before it reaches any controller. Each filter can inspect or modify the request, authenticate it, reject it, or let it continue.

11.2 SessionCreationPolicy.STATELESS

This tells Spring Security not to create or use server-side HTTP sessions for authentication state.

We need this because the auth service uses JWT. The token itself carries the authentication information, so a server session is unnecessary.

11.3 AuthenticationManager

`AuthenticationManager` is the entry point Spring Security uses to verify login credentials.

How It Is Used Here

`AuthServiceImpl.login()` sends a `UsernamePasswordAuthenticationToken` with email and password into the authentication manager.

Internal Working

```
AuthServiceImpl.login()
  -> AuthenticationManager.authenticate(...)
  -> delegates to configured AuthenticationProvider
  -> provider loads user details
  -> provider checks password with PasswordEncoder
  -> success returns authenticated token, failure throws exception
```

11.4 AuthenticationProvider and DaoAuthenticationProvider

`DaoAuthenticationProvider` is the Spring Security provider used here. It is designed for username/password login backed by a user store.

It is configured with:

- a `UserDetailsService` to load the user
- a `PasswordEncoder` to compare passwords safely

11.5 UserDetailsService and UserDetails

Spring Security does not use the JPA entity directly. It expects a `UserDetailsService` that can load a `UserDetails`.

- `CustomUserDetailsService` loads the user from the repository by email
- `AuthenticatedUser` adapts the app's `User` entity into Spring Security's `UserDetails` format

This separation is useful because the security layer may need concepts like authorities and account-enabled flags that are not part of a plain entity contract.

11.6 PasswordEncoder

The service uses `BCryptPasswordEncoder`.

Why It Is Needed

- Passwords must never be stored in plain text
- Hashing protects passwords if the database is leaked
- BCrypt includes salting and is designed for password storage

How It Is Used

- `encode(...)` during registration and password change

- `matches(...)` during password change checks and login verification

11.7 SecurityContextHolder

This is the central holder for the current authenticated user during a request.

The JWT filter writes authentication into it, and later MVC can read that identity back as a Principal.

12. Custom JWT Filter Concepts

12.1 OncePerRequestFilter

`JwtAuthenticationFilter` extends `OncePerRequestFilter`.

Definition

This is a Spring web filter base class that guarantees the filter runs once per request dispatch.

Why It Is Needed

JWT should be checked once per incoming request. Running the logic multiple times for the same request would be wasteful and could complicate behavior.

12.2 addFilterBefore(...)

The security configuration inserts the JWT filter before `UsernamePasswordAuthenticationFilter`.

This ordering matters. We need the JWT-based authentication to happen early enough that the security context is already ready before protected endpoints are resolved.

12.3 Internal Request Authentication Flow

```
Incoming request
  -> SecurityFilterChain starts
  -> JwtAuthenticationFilter checks Authorization header
  -> token is parsed and validated
  -> email claim is extracted
  -> user is loaded from database
  -> UsernamePasswordAuthenticationToken is built
```

```
   -> SecurityContextHolder is populated
   -> controller receives authenticated Principal
```

13. JWT Service Concepts Inside a Spring Boot App

JWT itself is not a Spring Boot concept, but the way this auth service integrates JWT into Spring Boot absolutely is.

What Spring Contributes Here

- Configuration binding for issuer, secret, and durations
- Bean lifecycle for `JwtTokenService`
- Filter-chain integration for access-token checks
- Controller/service wiring for refresh and validate flows

Why Access Token and Refresh Token Are Kept as Two Concepts

This service uses two token types so security and convenience can both be reasonable.

- **Access token** is for normal protected API calls
- **Refresh token** is only for getting a new token pair

The token type is enforced by custom claims and checked by `JwtTokenService.isTokenUsable(...)`.

14. Spring Boot Actuator

The service includes `spring-boot-starter-actuator` and exposes `health` and `info`.

Definition

Actuator provides operational endpoints used to observe and monitor a running service.

Why It Is Needed

- Health checks for deployment platforms and microservice environments
- Basic operational visibility

In a microservice architecture, Actuator health endpoints are especially important because service availability needs to be checked automatically.

15. Embedded Server Concept

Because the project uses `spring-boot-starter-web`, Spring Boot starts an embedded web server automatically. You do not need to deploy this service manually into an external Tomcat installation.

Why It Matters

- Simple development workflow
- Easier microservice deployment
- Each service can run independently on its own port

16. Spring Boot Testing Concepts

16.1 @SpringBootTest

`@SpringBootTest` starts a real application context for tests.

Why It Is Needed Here

The auth service includes security filters, repositories, validation, exception handling, and controller wiring. A narrow unit test would miss many integration points. A bootstrapped test ensures the whole stack works together.

16.2 @AutoConfigureMockMvc

This tells Spring Boot to configure `MockMvc`, which is a testing tool for simulating HTTP requests to the application without starting a real network server.

16.3 MockMvc

`MockMvc` is used to test:

- registration
- search
- login

- validate token
- refresh token
- change password
- deactivate account

Internal Working

```
Test method runs
  -> Spring Boot test context is already loaded
  -> MockMvc simulates HTTP request
  -> request goes through MVC and security stack
  -> controller/service/repository logic executes
  -> assertions check JSON and status
```

17. H2 and MySQL Integration Through Spring Boot

The service uses H2 by default and can switch to MySQL through a profile. This is a common Spring Boot pattern: keep local development simple, but make production-like storage easy to activate by configuration.

Boot auto-configures the datasource based on the active profile and the available JDBC driver.

18. Complete Internal Flow of the Auth Service in Spring Terms

```
Application startup
  -> @SpringBootApplication bootstraps context
  -> auto-configuration loads MVC, security, JPA, validation, actuator
  -> component scan finds controllers, services, repositories, filter, config
  -> beans are created and wired by dependency injection

Register request
  -> DispatcherServlet routes request to AuthResource
  -> @RequestBody converts JSON to RegisterRequest
  -> @Valid triggers DTO validation
  -> AuthServiceImpl performs business rules
  -> PasswordEncoder hashes password
  -> UserRepository persists entity through JPA
  -> JwtTokenService creates tokens
  -> ResponseEntity returns JSON response

Protected request
```

```
  -> SecurityFilterChain runs first
  -> JwtAuthenticationFilter authenticates request from Bearer token
  -> SecurityContextHolder stores identity
  -> Principal is resolved for controller method
  -> service method executes with authenticated user context

Failure path
  -> exception is thrown from service/controller
  -> @RestControllerAdvice catches it
  -> consistent HTTP error response is returned
```

19. Why These Spring Boot Concepts Are a Good Fit for This Service

Concept	Why it is a good fit for auth-service
Auto-configuration	Reduces infrastructure boilerplate so the service focuses on auth logic instead of low-level setup.
Dependency injection	Makes security, persistence, and configuration easy to connect without hard-coding object creation.
Spring MVC	Fits naturally because the service exposes a REST API.
Spring Security	Provides a production-grade authentication pipeline instead of custom login code.
Spring Data JPA	Makes user persistence and revoked-token persistence concise and maintainable.
Profiles and configuration properties	Let the same codebase work for local, MySQL, and OAuth-enabled modes with minimal friction.
Actuator	Supports operational health checks, which matter in microservice deployments.
Spring Boot testing	Verifies the real integration of controllers, filters, repositories, and config instead of only isolated units.

20. Final Summary

The auth service is a good example of how Spring Boot is not just “used to start the app.” In this project, Spring Boot and the wider Spring stack:

- start and configure the application
- scan and create beans automatically
- inject dependencies into the right classes
- bind configuration safely into Java objects
- handle web requests and JSON conversion
- validate input before business logic runs
- persist entities through repositories and transactions
- secure routes and authenticate requests
- provide operational health endpoints
- support end-to-end integration testing

In short, Spring Boot gives this auth-service its runtime skeleton, while Spring MVC, Security, Data JPA, Validation, Actuator, and Testing fill in the web, persistence, security, and operational parts of the service.

> This document is intentionally focused on the Spring Boot and Spring ecosystem concepts used in `auth-service` only. That means it is a service-specific learning guide, not a generic Spring Boot tutorial. We can create the same style of document for the next service one by one.